Problem 1(a). Faulty induction.
Induction proofs can break for many reasons. Careless summoning of the genie or misuse of the induction boilerplate may result in fake "proofs" for statements that are just plain false. Point out the most significant error from the following arguments and justify.
Let B be a set of blocks, arranged into stack(s). The blocks in each stack are in a linear order from bottom to top, with every block touching exactly one block from below and one from above (except for the top- and bottom-most blocks); two blocks from two separate stacks do not touch one another.
Consider the "proof" of the following statement: If every block in B is touching at least one other block in B, then all blocks in B are in a single stack.

> We prove the theorem by induction on the number of blocks in B. Let $P(B)$ be the statement "if every block in B is touching at least one other block in B, then all blocks in B are in a single stack."
> Consider an arbitrary set of blocks B arranged in stacks with exactly n blocks, where every block in B touches at least one other block in B. Assume as the induction hypothesis that $P(B')$ holds for any set of blocks B' with fewer than n blocks.
>
> - When B is empty, the statement "every block in B is touching at least one other block in B" is vacuously true, and "all blocks in B are in a single stack" is also vacuously true.
>
> - Otherwise, B is non-empty. Remove one block b from B to obtain a set B' containing $n-1$ blocks. By the induction hypothesis on B', all the blocks in B' are in a single stack. Because in $P(B)$ every block in B is assumed to touch at least one other block in B, block b must touch another block in B (which must be in B'). This means b is in the same stack as all blocks in B'. Therefore, all blocks in B are in a single stack.
>
> In both cases $P(B)$ holds, so the theorem holds.

Solution.

The statement that the proof proves is not necessarily true. The blocks in B can be arranged in 2 (or more, each with at least 2 blocks) stacks such that each block touches at least one other block, yet the blocks are not all in a single stack.

The proof specifically breaks when the author removes a block b from B. After removing b, we are no longer guaranteed if every block in B' touches at least one other block in B'.

Since the statement $P(B')$ requires that every block in B' touches at least one other block

in B', the induction hypothesis cannot be applied. Thus, the inductive step applies the induction hypothesis to a set that does not necessarily satisfy the assumptions of the original statement, and the proof is invalid.

Problem 1(b).
Recall the definition of big-O notation: the proposition $f(n) = O(g(n))$ means

$$\exists C > 0 \; \exists N \; \forall n \geq N, \quad f(n) \leq C \cdot g(n).$$

Let $\texttt{MS}(n)$ be defined by the following pseudocode:

```
MS(n):
    if n = 1: return 1
    else: return 2 * MS(⌊(n/2)⌋) + n
```

Consider the following "proof" a student submitted to show that $\texttt{MS}(n) = O(n)$.
We prove the statement by induction on n. Let n be an arbitrary positive integer, and assume that for every positive integer $m < n$, $\texttt{MS}(m) = O(m)$.

- If $n = 1$, then $\texttt{MS}(1) = 1 \leq C \cdot 1$ for any $C \geq 1$.
- If $n > 1$, then
$$\texttt{MS}(n) = 2 \cdot \texttt{MS}(\lfloor n/2 \rfloor) + n.$$
By the induction hypothesis, there exist constants $C' > 0$ and N' such that
$$\texttt{MS}(\lfloor n/2 \rfloor) \leq C' \cdot \lfloor n/2 \rfloor \quad \text{for all } n \geq N'.$$
Thus,
$$\texttt{MS}(n) \leq 2C' \cdot \lfloor n/2 \rfloor + n \leq C'n + n = (C' + 1)n.$$
Choosing $C \geq C' + 1$ gives $\texttt{MS}(n) \leq Cn$.

Therefore, $\texttt{MS}(n) = O(n)$.

Solution.

While invoking the induction hypothesis, the author misinterprets the existence of some constants C', N' for each $n > N'$, as the fact that for all $n > N$, there exist some constants C', N'. Which is false according to the definition of big-O notation. Hence, the induction hypothesis does not hold and the proof breaks.

The induction hypothesis is not true to the definition of big-O notation. The author states that for each integer $m < n$, the statement $\texttt{MS}(m) = O(m)$ holds. By definition of big-O, this means that for each such m there exist constants $C_m > 0$ and N_m such that

$$\texttt{MS}(k) \leq C_m k \quad \text{for all } k \geq N_m.$$

However, the proof assumes the existence of a single pair of constants C' and N' that works uniformly for $\texttt{MS}(\lfloor n/2 \rfloor)$, independent of n. Big-O notation does not guarantee that the same

constants can be chosen for all smaller inputs. The proof uses big-O notation incorrectly, assuming there is a fixed inequality $\texttt{MS}(m) \leq Cm$ holding uniformly for all m, which is stronger than what the definition of allows.

Thus the induction hypothesis is not correct, and the proof breaks.

Problem 2. Balanced parentheses.
A balanced parentheses string is a string over the symbols `[` and `]`, defined recursively as one of the following:

- the empty string ϵ;
- a string $[w]$ for some balanced parentheses string w;
- a string xy for some nonempty balanced parentheses strings x and y.

For example, `[[[][]]][][[[]]][]` is a balanced parentheses string of length 18.

1. Prove by induction that removing any pair of consecutive symbols `[]` (if one exists) from a balanced parentheses string results in another balanced parentheses string.

⋆ 2. Prove by induction that removing any pair of consecutive symbols `][` (if one exists) from a balanced parentheses string results in another balanced parentheses string.

Solution.

1. Let a be any balanced string of length n. Because a is recursively defined, we can write,

$$a = xy$$

for some balanced strings x, y. Note that if a is of the form

$$a = [w]$$

then without loss of generalization we let $x = \epsilon$, $y = [w]$. That is,

$$a = \epsilon\,[w] = xy.$$

Then, we can prove the claim by induction. Also note, that every balanced parentheses string that is not an empty string ϵ must start with `[`. This is because the second recursive definition is the only definition which represents a balanced string with the use of `[]`, in which case the `[` appears before `]`.

Assume that for any balanced string a' of length less than n, removing any pair of consecutive symbols `[]` (if one exists) from a' results in a balanced parantheses string. Then for the string $a = xy$ of length n, we have the following cases:

(a) $x = \epsilon$ or $y = \epsilon$

Without loss of generalization assume $y = \epsilon$. Then $a = x$. By the recursive definition, $a = x = [w]$ for some balanced parantheses string w.(Note that if y is of the form $y = x'y'$ for some balanced parantheses strings x' and y' then we can just assign $x = x'$, $y = y'$ which is then handled by the second case)

Since, w is a balanced parantheses string, we know that the first character is `[` and the last character is `]`. So, the sequence `[]` doesn't appear at the edges of a. Hence, we know that if the pair of consecutive symbols `[]` exists, it exists inside w. But since length of w is $n-2$, by the induction hypothesis we know that removing any pair of consecutive symbols `[]` (if one exists) from w results in a balanced parantheses string. Let the resultant balanced parantheses string be w'. Putting back the symbols `[]` spanning w, we get $[w']$, another balanced parantheses string. Hence, the induction holds.

(b) Neither x nor y is empty.

Then,

$$a = xy$$

where both x and y are balanced parantheses strings of length less than n. Since, x, y are balanced parantheses strings, we know that for each of them the first character is `[` and the last character is `]`. So, the sequence `[]` doesn't appear at the junction between x and y. Hence, we know that if the pair of consecutive symbols `[]` exists, it exists inside x or y.

Since both x, y are balanced parantheses strings of length less than n, by the induction hypothesis we know that that removing any pair of consecutive symbols `[]` (if one exists) from x, y results in balanced parantheses strings. And, since composition of balanced proposition strings is a balanced proposition string, the induction holds.

(c) $x = y = \epsilon$, $w = \epsilon$

This case is vacuously true because there exist no consecutive symbols `[]`.

2. Similarly, let a be any balanced string of length n. Because a is recursively defined, we can write,

$$a = xy$$

for some balanced strings x, y. Note that if a is of the form

$$a = [w]$$

then without loss of generalization we let $x = \epsilon$, $y = [w]$. That is,

$$a = \epsilon\,[w] = xy.$$

Then, we can prove the claim by induction. Assume that for any balanced string a' of length less than n, removing any pair of consecutive symbols `][` (if one exists) from a' results in a balanced parantheses string. Then for the string $a = xy$ of length n, we have the following cases:

(a) $x = \epsilon$ or $y = \epsilon$

Without loss of generalization assume $y = \epsilon$. Then $a = x$. By the recursive definition, $a = x = [w]$ for some balanced parantheses string w. (Note that if y is of the form $y = x'y'$ for some balanced parantheses strings x' and y' then we can just assign $x = x'$, $y = y'$ which is then handled by the second case)

Since, w is a balanced parantheses string, we know that the first character is `[` and the last character is `]`. So, the sequence `][` doesn't appear at the edges of a.

Hence, we know that if the pair of consecutive symbols `][` exists, it exists inside w. But since length of w is $n-2$, by the induction hypothesis we know that removing any pair of consecutive symbols `][` (if one exists) from w results in a balanced parantheses string. Let the resultant balanced parantheses string be w'. Putting back the symbols `[]` spanning w, we get $[w']$, another balanced parantheses string. Hence, the induction holds.

(b) Neither x nor y is empty.

Then,

$$a = xy$$

where both x and y are balanced parantheses strings of length less than n. Since, x, y are balanced parantheses strings, we know that for each of them the first character is `[` and the last character is `]`. So, the sequence `][` does appear at the edge of x and y. By the recursive definition of balanced parentheses strings, each of x and y can be written as a concatenation of balanced strings of the form $[w]$. That is,

$$x = [w_1][w_2]\cdots[w_k] \quad \text{and} \quad y = [w_{k+1}][w_{k+2}]\cdots[w_m],$$

where each w_i is a balanced parentheses string.

At the junction between x and y, the string contains the consecutive symbols `][` arising from the substring

$$[w_k][w_{k+1}].$$

Removing this consecutive pair `][` gives us the string

$$[w_k w_{k+1}],$$

which is balanced, since the concatenation $w_k w_{k+1}$ of two balanced parentheses strings is itself a balanced parentheses string by the recursive definition.

Any other occurrence of `][` lies entirely within x or within y, and since both x and y have length less than n, by the induction hypothesis we know that that removing any pair of consecutive symbols `][` (if one exists) from x, y results in balanced parantheses strings. And, since composition of balanced proposition strings is a balanced proposition string, the induction holds.

Hence, removing any pair of consecutive symbols `][` from a results in a balanced parentheses string in this case.

(c) $x = y = \epsilon$, $w = \epsilon$

This case is vacuously true because there exist no consecutive symbols `][`.

Problem 3. **Fibonacci representations.**
The Fibonacci numbers are defined by

$$F_n = \begin{cases} 0 & \text{if } n = 0, \\ 1 & \text{if } n = 1, \\ F_{n-1} + F_{n-2} & \text{otherwise.} \end{cases}$$

Prove that every non-negative integer can be written as a sum of distinct, non-consecutive Fibonacci numbers using the following algorithm:

```
TOFIBO(n):
    if n = 0: return <>
    else:
        F_l ← largest Fibonacci number less than n
        ⟨i_k, ..., i_1⟩ ←  TOFIBO n − F_l
        return ⟨l, i_k, ..., i_1⟩
```

In other words, no two consecutive Fibonacci numbers F_i and F_{i+1} appear in the sum. (Assume $F_0 = 0$ never appears.)
For example,

$$30 = F_8 + F_6 + F_2.$$

Solution.

We prove the correctness of the algorithm `TOFIBO` by induction. Let n be an arbitrary non-negative integer. Assume that `TOFIBO` works for all integers $m < n$. That is, every $m < n$ we can write m as a sum of distinct, non-consecutive Fibonacci numbers, such that

$$m = \sum_{i=1}^{k} F_i, \quad i \in \langle i_1, ..., i_k \rangle$$

where $\langle i_1, ..., i_k \rangle$ are non-consecutive indices of Fibonacci numbers arranged in increasing order, and i_k is the largest Fibonacci number less than m.

Then we have three cases:

1. n =0

   If n =0, the algorithm returns empty set$\langle\rangle$

2. $n > 0$ and is a Fibonacci number.

   In that case, we can write n as the singleton sum of n. That is, the largest Fibonacci number F_l such that $F_l \leq n$ is n, `TOFIBO` returns $\langle l \rangle$, and we're done.

3. $n > 0$ and not a Fibonacci number.

   In that case, there exists Fibonacci numbers less than n. Pick the largest such Fibonacci number, call it F_l. Then,

   $$n - F_l = n'$$

   for some n'. Since both n and F_l are nonnegative integers, n' must be a nonnegative integer less than n. By our induction hypothesis n' can be written as a sum of distinct, non-consecutive Fibonacci numbers,

   $$n' = \sum_{i=1}^{k} F_i, \quad i \in \langle i_1, ..., i_k \rangle$$

   We know that F_l and F_{i_k} are not consecutive Fibonacci numbers because if they were, then $F_{i_k} = F_{l-1}$. In that case,

   $$n \geq F_l + F_{l-1}$$

   But $F_l + F_{l-1}$ is also a Fibonacci number, specifically F_{l+1}. That is,

   $$n \geq F_{l+1}$$

   which means F_l isn't the largest Fibonacci number less than n, which contradicts our choice of F_l as the largest Fibonacci number less than n. Hence, we can write n as a sum of distinct, non-consecutive Fibonacci numbers,

   $$n = F_l + n' = F_l + \sum_{i=1}^{k} F_i \quad i \in \langle i_1, ..., i_k \rangle$$

   Therefore, `TOFIBO` returns $\langle i_1, \ldots, i_k, l \rangle$.

Problem 4*. Ludicrous tiling.
Let Q be the first quadrant of the real plane $\mathbb{R}^2$. You have an unlimited supply of L-shaped tiles, each consisting of a 2×2 square with one unit square removed.
Prove that you can tile the entire quadrant Q using only L-shapes and backward-L-shapes, with no rotations allowed.

Problem 5*. Rooted binary tree.
A rooted binary tree is defined recursively as either:

- the empty tree, denoted `null`; or
- a root node $T.x$, a left subtree T_ℓ, and a right subtree T_r, where T_ℓ and T_r are rooted binary trees.

In C, such a structure can be defined as:

```
struct binaryTree {
    int root;
    struct binaryTree *leftSubtree;
    struct binaryTree *rightSubtree;
};
```

Let T be a rooted binary tree with n vertices. Deleting any node v splits T into at most three subtrees: one containing the left child of v, one containing the right child of v, and one containing the parent of v (if they exist).
We call v a *central node* if each resulting subtree has at most $n/2$ nodes. Prove that every rooted binary tree has at least one central node.